Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

Taller 2

Objetivos

- Implementar algoritmos de ordenamiento en distintos escenarios.
- Analizar la complejidad temporal segun el tamaño de los datos.
- Modificar algoritmos de sorting para resolver problemas especificos.

Introducción

Después de varias semanas ayudando al Profesor Oak a organizar información sobre los Pokémon, ha surgido un nuevo desafío en el laboratorio. Investigadores de distintas regiones han comenzado a enviar grandes cantidades de datos sobre Pokémon, incluyendo atributos como nivel, velocidad y ataque, entre muchos otros.

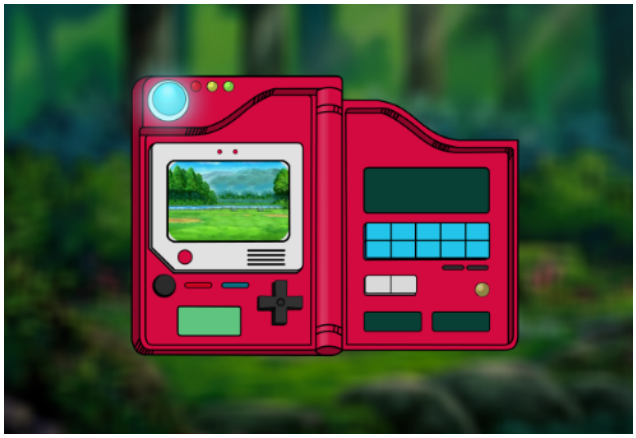
El problema es que toda esta información llega desordenada, y organizarla manualmente podría tomar demasiado tiempo. Para poder analizar correctamente los datos y continuar con las investigaciones, es necesario implementar métodos eficientes que permitan ordenar grandes volúmenes de información de manera rápida.

En este taller, su tarea como ayudante del laboratorio será implementar distintos algoritmos de ordenamiento que permitan organizar los datos de los Pokémon según distintos criterios. A través de estos ejercicios, se busca analizar el comportamiento de distintos algoritmos de sorting y comprender cuándo es conveniente utilizar cada uno de ellos dependiendo del tamaño y características de los datos.

¡Orden en la Pokedex!

En el laboratorio del Profesor Oak se reciben constantemente nuevos registros de Pokemon provenientes de distintas regiones. Cada investigador envía información sobre los Pokemon que ha observado, incluyendo atributos como su ID en la pokedex, su nivel y su velocidad.

Debido a la gran cantidad de datos recibidos, la información llega completamente desordenada. Para poder analizar correctamente estos registros y compararlos entre distintas investigaciones, el laboratorio necesita ordenar los datos de forma eficiente.



Problema

El Profesor Oak ha reunido varios registros de Pokemon en arreglos de datos. Dependiendo del tamaño del conjunto de datos, distintos algoritmos de ordenamiento pueden ser más o menos eficientes.

Utilizando tus conocimientos de programación, deberás implementar distintos algoritmos de sorting para ordenar los registros del laboratorio y facilitar su análisis.

Entidades

Pokemon: Cada Pokémon cuenta con un `ID` único, un valor de `nivel` y un valor de `velocidad`. Todos los atributos son valores numéricos.

Flujo

Primero recibirás una línea con el entero E, que representa la cantidad de eventos a procesar.

Luego se recibirán E eventos, los cuales corresponden a distintas operaciones que el laboratorio del Profesor Oak necesita realizar sobre los registros de Pokemon.

Cada evento tendrá el nombre que indica el tipo de operación a ejecutar. Dependiendo del tipo de evento, se entregará la información necesaria para realizar la operación correspondiente.

Los eventos deberán ser procesados en el mismo orden en que aparecen en el input.

Para los eventos que requieran ordenar información, recibirás un entero N que representa la cantidad de Pokémon a procesar, seguido de N líneas con la información correspondientes al Pokemon.

Dependiendo del tipo de evento, deberás aplicar el algoritmo de ordenamiento correspondiente y mostrar el resultado solicitado.

Parte Formativa - Eventos

`SMALL_SORT {N} {atr} {orden}`

En este evento deberás ordenar los N Pokémon que se te entregaran según el atributo indicado en `atr`, que puede ser `NIVEL` o `VELOCIDAD`. Junto a el parámetro `orden`, que puede ser `ASC` o `DESC`, indicando si el ordenamiento debe realizarse de forma ascendente o descendente. Todo esto siguiendo con N lineas de los Pokemon a registrar con sus respectivos `ID`, `NIVEL`, `VELOCIDAD`.

En caso de empate en ambos atributos seleccionados, deberás desempatar con respecto al orden entregado utilizando el `ID` del Pokémon, el cual es único.

Este evento corresponde a conjuntos de datos pequeños, por lo que se espera que utilices un algoritmo de ordenamiento con complejidad aproximada $O(n^2)$.

El resultado esperado debe mostrar el siguiente output:

output

```
{N} Pokemones ordenados por {atr} en forma {orden}:
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
    .
    .
    .
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
```

Donde atr podria ser tanto VELOCIDAD como NIVEL segun el caso.

`BIG_SORT {N} {atr_1} {atr_2} {orden}`

En este evento deberás ordenar los N Pokémon que se te entregaran según el primer atributo indicado en `atributo_1`, que puede ser `NIVEL` o `VELOCIDAD`. En caso de empate con el primer atributo se debera hacer la comparacion con el segun atributo `atributo_2`, que tambien puede ser `NIVEL` o `VELOCIDAD`. Todo junto a el parámetro `orden`, que puede ser `ASC` o `DESC`, indicando si el ordenamiento debe realizarse de forma ascendente o descendente. FInalmente siguiendo con N lineas de los Pokemon a registrar con sus respectivos `ID`, `NIVEL`, `VELOCIDAD`.

En caso de empate en ambos atributos seleccionados, deberás desempatar con respecto al orden entregado utilizando el `ID` del Pokémon, el cual es único.

Este evento corresponde a conjuntos de datos grandes, por lo que se espera que utilices un algoritmo de ordenamiento con complejidad aproximada $O(n * logn)$ para que sea eficiente.

El resultado esperado debe mostrar el siguiente output:

output

```
{N} Pokemones ordenados por {atr\_1} {atr\_2} en orden {orden}:
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    .
    .
    .
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
```

Se puede suponer que `atr_1` y `atr_2` siempre seran distintos.

Parte Evaluada - Grupo 1

`ANAGRAM {N} {nombre_1} {nombre_2}`

Un anagrama se conoce como la reordenación de las letras de una palabra para formar una nueva, utilizando exactamente las mismas letras el mismo número de veces.

El Profesor Oak desea analizar similitudes entre nombres. Por lo que en este evento recibirá un `N` que indica el largo de estos strings, junto a dos nombres, `nombre_1` y `nombre_2`, y tiene como objetivo determinar si sus nombres son anagramas entre sí.

Puede asumir que los nombres seran de igual largo y estaran solamente constituidos por letras en minusculas y no superarán los 50 caracteres, por lo que no es necesario considerar numeros, mayusculas y signos en la comparacion.

output si es verdadero el evento:

```
    {nombre_1} si es anagrama de {nombre_2}.
```

output si es falso el evento:

```
    {nombre_1} no es anagrama de {nombre_2}.
```

HINT: Si defines `char string1[N]` e `int array_int[N]`, entonces al hacer $array_int[i] = string1[i]$ estás guardando el carácter en la posición i, pero como su valor representado en un int. Para este evento, contaran con la funcion *merge_ints* para poder ordenar ints via mergesort.

`HYBRID_SORT {N} {K} {atr_1} {atr_2} {orden}`

En el mundo Pokemon, el Profesor Oak desea ordenar un arreglo de `N` pokemones utilizando una estrategia híbrida de ordenamiento. Para esto, en el taller formativo, implementaste las funciones small sort y Big sort, las cuales usaban los algoritmos de Insertion sort y Mergesort respectivamente, ahora tu trabajo es crear una nueva función Hybrid sort, que fusione los algoritmos de InsertionSort y MergeSort en uno solo.

Por ejemplo, si tengo un arreglo de tamaño $N = 10$ y $K = 5$, primero se usaria **MERGE_SORT**, ya que $N > K$, pero, por como funciona **MERGE_SORT**, este dividirá el array de tamaño 10, en dos subarreglos de tamaño 5 y ordenara esos dos de forma recursiva. Finalmente como ambos de estos arreglos tienen un $N \leq K$, entonces ambos deben ser ordenados con **INSERTION_SORT**. Por lo tanto, **HYBRID_SORT** seria la fusion de los algoritmos insertion sort y merge sort para ser usados segun corresponda.

Cada vez que se ejecute una llamada a `MERGE_SORT` o `INSERTION_SORT`, se debe imprimir un mensaje indicando el algoritmo utilizado y el tamaño del arreglo sobre el cual se está trabajando hasta terminar la recursividad.
Tips:

- Considere el siguiente llamado a la función insertionsort trabajada en el taller formativo:

$$Insertionsort(\&pokemones[start], (end - start - 1), atr1, atr2, orden)$$

  De esta forma, le dices a **InsertionSort** que ordene en el arreglo pokemones, por prioridad en el atributo atr1, luego atr2 en caso de empate y orden, iniciando en la posición start del arreglo y ordenando exactamente (end - start - 1) elementos después de este inicio.

- Ademas agregar un nuevo caso en **InsertionSort** que, dado empate en el atr1, que se resuelve respecto al atr2 y finalmente respecto a su ID que es unico.

.

output

```
Ejecutando MERGE_SORT en arreglo de largo {N}...
Ejecutando MERGE_SORT en arreglo de largo {N/2}...
Ejecutando INSERTION_SORT en arreglo de largo {K}...
.
.
Fin de la ejecucion.
{N} Pokemones ordenados por : {atr_1} {atr_2} en {orden}:
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    .
    .
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
```

BONUS

Finalmente, el Profesor Oak los quiere incentivar a que sigan buenas practicas, es por esto que les comenta que si logran cumplir con las tareas dadas **sin leaks ni errores de memoria**, seran premiados con puntos extra! **Hint: Recuerden ocupar la mausquerramienta misteriosa llamada valgrind!**

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `pokemones` que se ejecuta con el siguiente comando:

```
./pokemones input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind deberás utilizar el siguiente comando:

```
valgrind ./pokemones input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

El archivo de input comenzará con el número `E` que corresponde al número de eventos a recibir, y las `E` líneas correspondientes a cada uno. Para cualquiera de los dos eventos, se entregara tambien un numero `N`, que puede indicar la cantidad de pokemones a registrar a continuacion o el largo de las palabras dadas segun el evento.

Como ejemplo tenemos el siguiente input:

input

```
3 # Cantidad de eventos
HYBRID_SORT {5} {3} {VELOCIDAD} {NIVEL} {ASC}
1 11 12 # primer pokemon a registrar de los 5
5 12 10
4 1 5
6 56 3
10 5 8 # ultimo pokemon a registrar de los 5
ANAGRAM 8 animales milanesa
ANAGRAM 5 perro gatos
```

Output

output

```
Ejecutando MERGE_SORT en arreglo de largo 5...
Ejecutando INSERTION_SORT en arreglo de largo 2...
Ejecutando INSERTION_SORT en arreglo de largo 3...
Fin de la ejecucion.
5 Pokemones ordenados por VELOCIDAD NIVEL en orden ASC:
    Pokemon ID 1: VELOCIDAD 12 NIVEL 11
    Pokemon ID 5: VELOCIDAD 10 NIVEL 12
    Pokemon ID 10: VELOCIDAD 8 NIVEL 5
    Pokemon ID 4: VELOCIDAD 5 NIVEL 1
    Pokemon ID 6: VELOCIDAD 3 NIVEL 56
animales si es anagrama de milanesa.
perro no es anagrama de gatos.
```

Evaluación

La nota se calcula usando testcases especificos de evaluacion segun: 1 punto por output correcto y 0 puntos por output incorrecto.

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 2 segundos** y utilizar menos de 1 GB de RAM[1], de lo contrario recibirás 0 puntos en ese test. Por último, está **estrictamente prohibido** el uso de la función `realloc`, cuyo uso implicará un descuento de 5 décimas.

Integridad académica e IA

Cualquier acto deshonesto o fraude academico esta estrictamente prohibido. Se prohibe el uso de herramientas de inteligencia artificial y el uso o manipulacion de otros dispositivos electronicos como laptops, tablet, telefonos durante la evaluacion.

[1] Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`